Design Principles Behind `T2G2Pipeline`

Slide 1: Single Responsibility Principle (SRP)

> "Each module/class should have one and only one reason to change."

- `T2G2Pipeline` : manages end-to-end pipeline flow
- `Timeseries` : encapsulates time series data structure and metadata
- `preprocess_strategy.py` : focuses only on preprocessing
- `ts2g_strategy.py` : handles TS-to-graph transformation
- `embedding_strategy.py` : computes graph embeddings

Slide 2: Open/Closed Principle (OCP)

> "Software entities should be open for extension, but closed for modification."

- Strategy registries (e.g., `@register_strategy`) enable adding new methods
- No need to modify core pipeline when extending functionality

Slide 3: Liskov Substitution Principle (LSP)

> "Derived classes must be substitutable for their base classes."

- Interface expectations:
  - `.to_graph()` for TS-to-graph conversion
- Allows swapping strategies with minimal coupling

Slide 4: Interface Segregation Principle (ISP)

> "Clients should not depend on interfaces they do not use."

- `TimeGraph` defines only store the graphs, not the draw function or other state s
- Keeps interfaces lean and purpose-specific

Slide 5: Dependency Inversion Principle (DIP)

> "High-level modules should not depend on low-level modules."

- `T2G2Pipeline` accepts strategies as arguments (inversion of control)
- Does not depend on any hardcoded implementation

Slide 6: Summary Table

Principle	Implementation
SRP	Modular design: preprocessing, embedding, etc.
OCP	Easily extensible via registration
LSP	Swappable components via standard interfaces
ISP	Minimal, clean method contracts
DIP	Pipeline depends on abstraction, not concrete code

Slide 7: Optional Enhancements

- Introduce abstract base classes (e.g., `AbstractGraphBuilder`)
- Configurable CLI entry (e.g., via `argparse`)
- Strategy injection via plugin loaders
- Provide default unit tests using `pytest` or `unittest`